

```
print("\x1b[2J\033[H")
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
df=pd.read_csv(r"C:\Users\acer\OneDrive\Desktop\online_advertising_performance_data.csv")
```

```
pd.set_option("display.max_rows",None)
```

```
pd.set_option("display.max_columns",None)
```

```
print(df.info(),"\n-----\n")
```

```
print(df.nunique(),"\n-----\n")
```

```
print(df.isnull().sum(),"\n-----\n")
```

```
print(df['placement'].value_counts(),"\n-----\n")
```

```
df['placement'] = df['placement'].fillna(df['placement'].mode().iloc[0])
```

```
#Q1
```

```
user_engagement=df['user_engagement'].value_counts()
```

```
print(user_engagement,"\n-----\n")
```

```
#monthwise grouping
```

```
print(df.groupby(['month','campaign_number','user_engagement'])['user_engagement'].count(),"\n-----\n")
```

```
sns.catplot(x='user_engagement',hue='campaign_number', col='month',kind='count',data=df)
```

```
plt.show()
```

```
#Q2
```

```
print(df.groupby('banner')['clicks'].sum(),"\n-----\n")
```

```

clicks_by_size =
df.groupby('banner')['clicks'].sum().reset_index().sort_values(by='clicks',ascending=False)
#reset_index creates a dataframe with it. else it is a series

sns.barplot(data=clicks_by_size, x='banner', y='clicks')

plt.title("Total Clicks by Banner Size")

plt.xlabel("Banner Size")

plt.xticks(rotation=90)

plt.yticks([0, 200000, 400000, 600000, 800000, 1000000, 1200000])

plt.ylabel("total clicks")

plt.show()

```

```

avg_clicks = df.groupby('banner')['clicks'].mean().sort_values(ascending=False)

sns.barplot(x=avg_clicks.index, y=avg_clicks.values)

plt.title("Average Clicks per Ad by Banner Size")

plt.xlabel("Banner Size")

plt.ylabel("Average Clicks")

plt.xticks(rotation=90)

plt.tight_layout()

plt.show()

```

#Q3

```

print(df.groupby('placement')['clicks'].sum().sort_values(ascending=False),"\n-----\n")

```

```

print(df.groupby('placement')['clicks'].mean().sort_values(ascending=False),"\n-----\n")

```

```

print(df.groupby('placement')['displays'].sum().sort_values(ascending=False),"\n-----\n")

```

```

print(df.groupby('placement')['displays'].mean().sort_values(ascending=False),"\n-----\n")

```

```

avg_clicks_placement = df.groupby('placement')['clicks'].mean().sort_values(ascending=False)
sns.barplot(x=avg_clicks_placement.index, y=avg_clicks_placement.values)
plt.title("Average Clicks per Ad by Placement")
plt.xlabel("Placement")
plt.ylabel("Average Clicks")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()

```

```

avg_displays_placement = df.groupby('placement')['displays'].mean().sort_values(ascending=False)
sns.barplot(x=avg_displays_placement.index, y=avg_displays_placement.values)
plt.title("Average Displays per Ad by Placement")
plt.xlabel("Placement")
plt.ylabel("Average Displays")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()

```

#Q4

```

a=df.select_dtypes(include=['int64','float64'])
s=a.corr()
plt.figure(figsize=(10,8))
sns.heatmap(s,annot=True,cmap='coolwarm',fmt='0.2f')
plt.show()
print("Shows high correlation","\n-----\n")
#HIGH CORRELATION

```

#Q5

```

avg_rpc = df['revenue'].sum() / df['clicks'].sum()

```

```
print(f"Overall Average Revenue per Click: {avg_rpc:.4f}", "\n-----\n")
```

#Q6

```
print(df.groupby('campaign_number')['post_click_conversions'].sum())
print("camp 1 has most post click conversions ", "\n-----\n")
```

#Q7

```
grouped_sums=df.groupby('month')['post_click_sales_amount'].sum().reset_index()
sns.barplot(data=grouped_sums, x='month', y='post_click_sales_amount')
plt.title("post click Sales over time")
plt.xticks(rotation=90)
plt.show()
```

```
df['time'] = pd.to_datetime("2020-" + df['month'] + "-" + df['day']).astype(str, format="%Y-%B-%d")
```

```
grouped_sums=df.groupby('time')['post_click_sales_amount'].sum().reset_index()
sns.lineplot(data=grouped_sums, x='time', y='post_click_sales_amount')
plt.title("post click Sales over time")
plt.xticks(rotation=90)
plt.show()
```

#Q8

```
print(df['banner'].value_counts())
print(df.groupby(['banner', 'user_engagement'])['user_engagement'].count(), "\n-----\n")
sns.countplot(x='banner', hue='user_engagement', data=df)
plt.title("User Engagement compared to each banner sizes")
plt.xticks(rotation=90)
plt.show()
```

#Q9

```
print("Placements by post click conversion rates\n")
```

```
print(df.groupby('placement')['post_click_conversions'].sum().sort_values(ascending=False))

print("GHI has highest post click conversion rates", "\n-----
\n")
```

#Q10

```
grouped_sums1=df.groupby('time')['clicks'].sum().reset_index()
sns.lineplot(data=grouped_sums1, x='time', y='clicks')
plt.title("Clicks over time")
plt.xticks(rotation=90)
plt.show()
print("Clicks decrease over time\n")
```

```
grouped_sums2=df.groupby('time')['displays'].sum().reset_index()
sns.lineplot(data=grouped_sums2, x='time', y='displays', color='Red')
plt.title("Displays over time")
plt.xticks(rotation=90)
plt.show()
print("Displays also decrease over time\n")
print("\n-----\n")
```

#Q11

```
df['engagement_score'] = df['user_engagement'].map({'Low': 1, 'Medium': 2, 'High': 3})
correlation = df[['engagement_score', 'revenue']].corr().iloc[0, 1]
print(f"Correlation between engagement and revenue: {correlation:.3f}")
print("\n-----\n")
```

#Q12

```
#outliers using boxplot for sum in each day
A1=df.groupby('time')['clicks'].sum().reset_index()
```

```
print(A1['clicks'].describe(), "\n-----\n")
sns.boxplot(y='clicks', data=A1)
plt.title('Box plot for clicks(total sum per day)')
plt.show()
```

```
A2=df.groupby('time')['cost'].sum().reset_index()
print(A2['cost'].describe(), "\n-----\n")
sns.boxplot(y='cost', data=A2)
plt.title('Box plot for cost(total sum per day)')
```

```
plt.show()
A3=df.groupby('time')['revenue'].sum().reset_index()
print(A3['revenue'].describe(), "\n-----\n")
sns.boxplot(y='revenue', data=A3)
plt.title('Box plot for revenue (total sum per day)')

plt.show()
```

#for individual data points

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
```

```
sns.boxplot(y='cost', data=df, ax=axes[0], color='lightblue')
axes[0].set_title("Cost")
```

```
sns.boxplot(y='clicks', data=df, ax=axes[1], color='lightgreen')
axes[1].set_title("Clicks")
```

```
sns.boxplot(y='revenue', data=df, ax=axes[2], color='salmon')
axes[2].set_title("Revenue")
```

```
plt.tight_layout()
```

```
plt.show() #Shows huge number of outliers, if counting per data
```

```
#Q13
```

```
#Effectiveness of campaign is shown by the quantities, user engagement and post click income
```

```
#User engagement v/s placements
```

```
print(df.groupby(['placement','user_engagement']).size(), "\n-----  
-----\n")
```

```
sns.countplot(x='placement',hue='user_engagement',data=df)
```

```
plt.title("User Engagement compared to each placement type")
```

```
plt.xticks(rotation=90)
```

```
plt.show()
```

```
print(df.groupby(['banner','user_engagement'])['user_engagement'].count(), "\n-----  
-----\n")
```

```
sns.countplot(x='banner',hue='user_engagement',data=df)
```

```
plt.title("User Engagement compared to each banner sizes")
```

```
plt.xticks(rotation=90)
```

```
plt.show()
```

```
sns.catplot(x='placement',hue='user_engagement',col="campaign_number",kind="count",data=df,palette="magma")
```

```
plt.title("User Engagement compared to each placement types across each camp")
```

```
plt.xticks(rotation=90)
```

```
plt.show()
```

```
sns.catplot(x='banner',hue='user_engagement',col="campaign_number",kind="count",data=df,palette="plasma")
```

```
plt.title("User Engagement compared to each Banner size across each camp")
```

```
plt.xticks(rotation=90)
```

```
plt.show()
```

#Q14

```
df = df[df['cost'] > 0].copy()
```

```
df['ROI'] = (df['revenue'] - df['cost']) / df['cost'] #Return on Investment.
```

```
roi_by_campaign = df.groupby('campaign_number')['ROI'].mean().sort_values(ascending=False)
```

```
print("Average ROI by Campaign:")
```

```
print(roi_by_campaign)
```

```
roi_by_banner = df.groupby('banner')['ROI'].mean().sort_values(ascending=False)
```

```
print("\nAverage ROI by Banner Size:")
```

```
print(roi_by_banner)
```

```
sns.barplot(x=roi_by_campaign.index, y=roi_by_campaign.values)
```

```
plt.title("Average ROI by Campaign")
```

```
plt.ylabel("ROI")
```

```
plt.xlabel("Campaign")
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
sns.barplot(x=roi_by_banner.index, y=roi_by_banner.values)
```



```
plt.title("Average ROI by Banner Size")
plt.ylabel("ROI")
plt.xlabel("Banner Size")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

#Q15

```
print("\n-----\n")
print("Post click conversions per placement")
print(df.groupby('placement')['post_click_conversions'].sum().sort_values(ascending=False))
```

#Q16

```
print("\n-----\n")
```

```
df['time'] = pd.to_datetime(df['time'])
df['day_name'] = df['time'].dt.day_name()
df['day_type'] = df['day_name'].apply(lambda x: 'Weekend' if x in ['Saturday', 'Sunday'] else
'Weekday')
print(df.groupby('day_type')['user_engagement'].value_counts().sort_values(ascending=False))
```

```
sns.countplot(data=df, x='user_engagement', hue='day_type', palette='Set2')
plt.title("User Engagement: Weekdays vs Weekends")
plt.ylabel("Count")
plt.xlabel("User Engagement Level")
plt.show()
```

#Q17

```
print("\n-----\n")
```

```

df = df[df['clicks'] > 0].copy()
df['CPC'] = (df['cost']/df['clicks']) #defining cost per click

print(df.groupby('campaign_number')['CPC'].sum().sort_values(ascending=False))

print(df.groupby('banner')['CPC'].sum().sort_values(ascending=False))

print("\n-----\n")

```

```

#JUST FOR FUN, CALCULATING THE REVENUE PER CLICK TOOOO
df['RPC'] = (df['revenue']/df['clicks']) #defining Revenue per click

print(df.groupby('campaign_number')['RPC'].sum().sort_values(ascending=False))

print(df.groupby('banner')['RPC'].sum().sort_values(ascending=False))

```

```

#Q18
print("\n-----\n")
df = df[df['cost'] > 0].copy() # Avoid divide-by-zero
df['effectiveness'] = df['post_click_conversions'] / df['cost']

print(df.groupby('campaign_number')['effectiveness'].sum().sort_values(ascending=False))
print(df.groupby('placement')['effectiveness'].sum().sort_values(ascending=False))

```

```

#Q19
print("\n-----\n")

print(df.groupby('day_name')['post_click_conversions'].sum().sort_values(ascending=False))

```

```

conversions_by_day = df.groupby('day_name')['post_click_conversions'].sum().reindex([
    'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'
])

sns.lineplot(x=conversions_by_day.index, y=conversions_by_day.values)
plt.title("Total Post-Click Conversions by Day of the Week")
plt.xlabel("Day of Week")
plt.ylabel("Post-Click Conversions")
plt.xticks(rotation=60)
plt.show()

```

#Q20

```

print("\n-----\n")

```

```

print(df.groupby(['campaign_number',
    'user_engagement'])['post_click_conversions'].sum().unstack(fill_value=0))

```

```

df.groupby(['campaign_number',
    'user_engagement'])['post_click_conversions'].sum().unstack(fill_value=0).plot(kind='bar',
    colormap='tab20')

```

```

plt.title("Post-Click Conversions by Campaign and User Engagement")
plt.xlabel("Campaign")
plt.ylabel("Post-Click Conversions")
plt.legend(title="User Engagement")
plt.tight_layout()
plt.show()

```